

Competitive Programming

Section 20 Editorial: 0-1 BFS And Shortest Path Variants

Competitive Programming Course

2026-10-01

Table of contents

1	Editorial overview	4
2	0-1 BFS reminder	5
3	A. Binary Weight Routes	6
3.1	Restatement	6
3.2	Observations	6
3.3	Algorithm	6
3.4	Correctness proof	6
3.5	Complexity	6
3.6	Full C++ solution	7
3.7	Full Python solution	8
4	B. Conveyor Grid	9
4.1	Restatement	9
4.2	Observations	9
4.3	Algorithm	9
4.4	Correctness	10
4.5	Complexity	10
4.6	Full C++ solution	10
4.7	Full Python solution	11
5	C. Warp Maze	12
5.1	Restatement	12
5.2	Observations	12
5.3	Algorithm	12
5.4	Correctness	13
5.5	Complexity	13
5.6	Full C++ solution	13
5.7	Full Python solution	14
6	Common wrong answers	16
7	Stress testing	17
8	What to carry forward	18
9	D. Letter Portals	19
9.1	Restatement	19
9.2	Model the implicit graph	19
9.3	Expanding one portal group once	19
9.4	Walk-through	19
9.5	Algorithm	20
9.6	Correctness proof	21
9.7	Complexity	21

9.8	Implementation notes	21
9.9	Full C++ solution	22
9.10	Full Python solution	23
9.11	Common wrong answers	24
9.12	Tests that matter	24
10	Stress testing	25
11	What to carry forward	26

1 Editorial overview

Problem	Model
A. Binary Weight Routes	directed graph with 0/1 edges
B. Conveyor Grid	direction-following costs 0, changing direction costs 1
C. Warp Maze	walking costs 0, warping costs 1

2 0-1 BFS reminder

For every relaxation:

```
if edge cost is 0: push_front  
if edge cost is 1: push_back
```

This preserves the same order that Dijkstra's priority queue would produce for distances that only differ by zero or one at each edge.

3 A. Binary Weight Routes

3.1 Restatement

Given a directed graph with edge weights 0 or 1, answer shortest distances from a fixed source to several targets.

3.2 Observations

All queries share one source, so one 0-1 BFS computes all answers.

3.3 Algorithm

```
dist[s] = 0
dq = [s]
while dq not empty:
    u = pop front
    for u -> v with weight w:
        if dist[u] + w < dist[v]:
            update dist[v]
            push front if w=0 else back
```

3.4 Correctness proof

Lemma. Whenever a state is relaxed with a smaller distance, inserting it at the front for cost 0 and back for cost 1 preserves nondecreasing distance order in the deque.

Proof. A cost-0 relaxation has the same distance as the currently popped state, so it belongs with the current smallest layer. A cost-1 relaxation is exactly one layer later, so it belongs after all states of the current layer. $\square$

Theorem. The algorithm computes shortest distances.

Proof. By the lemma, 0-1 BFS processes states in the same distance order as Dijkstra would for weights 0 and 1. Dijkstra's nonnegative-weight correctness therefore applies. $\square$

3.5 Complexity

Time: $O(n + m)$
Memory: $O(n + m)$

3.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s, qn;
    if (!(cin >> n >> m >> s >> qn)) return 0;
    --s;
    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u - 1].push_back({v - 1, w});
    }
    vector<int> queries(qn);
    for (int &x : queries) { cin >> x; --x; }

    const int INF = 1e9;
    vector<int> dist(n, INF);
    deque<int> dq;
    dist[s] = 0;
    dq.push_back(s);
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto [v, w] : adj[u]) {
            int nd = dist[u] + w;
            if (nd < dist[v]) {
                dist[v] = nd;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    for (int i = 0; i < qn; i++) {
        if (i) cout << ' ';
        cout << (dist[queries[i]] == INF ? -1 : dist[queries[i]]);
    }
    cout << '\n';
    return 0;
}
```

3.7 Full Python solution

```
from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it); m = next(it); s = next(it) - 1; qn = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1; v = next(it) - 1; w = next(it)
        adj[u].append((v, w))
    queries = [next(it) - 1 for _ in range(qn)]
    inf = 10**18
    dist = [inf] * n
    dist[s] = 0
    dq = deque([s])
    while dq:
        u = dq.popleft()
        for v, w in adj[u]:
            nd = dist[u] + w
            if nd < dist[v]:
                dist[v] = nd
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    print(" ".join(str(-1 if dist[x] == inf else dist[x]) for x in queries))

if __name__ == "__main__":
    main()
```

4 B. Conveyor Grid

4.1 Restatement

Moving in the direction written in the current cell is free; moving in another valid direction costs one. Find the minimum cost from top-left to bottom-right.

4.2 Observations

Each cell has up to four outgoing edges. The edge cost is either 0 or 1, so use 0-1 BFS.

Use the cell you are **leaving** to price a move. At (r, c) , the transition to the neighbor in direction `dir` has cost 0 exactly when `grid[r][c] == dir`; otherwise it costs 1. Reversing that lookup to inspect the destination cell solves a different problem.

State	Transition	Weight	Deque action after an improvement
(r, c)	valid neighbor in its printed direction	0	push front
(r, c)	any other valid neighbor	1	push back

For the sample, $S=(0,0)$ has R, so $(0,1)$ gets distance 0 and goes to the front. The free route initially follows right, right, down; when it must leave the indicated arrow it pays one. The answer is therefore a count of changed directions, not a count of moves.

4.3 Algorithm

```
dist[*][*] = infinity
dist[0][0] = 0
push (0, 0) to the front of a deque

while the deque is nonempty:
    (r, c) = pop front
    for every valid cardinal neighbor (nr, nc) with label dir:
        w = 0 if grid[r][c] == dir else 1
        if dist[r][c] + w < dist[nr][nc]:
            dist[nr][nc] = dist[r][c] + w
            if w == 0: push (nr, nc) to the front
            else:      push (nr, nc) to the back

print dist[n-1][m-1]
```

4.4 Correctness

Every grid route corresponds to a path in this 0/1 graph with the same cost, and every graph path is a valid route. By 0-1 BFS correctness, the distance to the bottom-right cell is the optimum.

4.5 Complexity

There are $n*m$ states and at most four transitions per state. 0-1 BFS runs in $O(n*m)$ time and uses $O(n*m)$ memory.

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<string> grid(n);
    for (auto &row : grid) cin >> row;
    const int INF = 1e9;
    vector<vector<int>> dist(n, vector<int>(m, INF));
    deque<pair<int,int>> dq;
    dist[0][0] = 0;
    dq.push_back({0, 0});
    int dr[4] = {1, -1, 0, 0};
    int dc[4] = {0, 0, -1, 1};
    char ch[4] = {'D', 'U', 'L', 'R'};
    while (!dq.empty()) {
        auto [r, c] = dq.front(); dq.pop_front();
        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            int w = (grid[r][c] == ch[i] ? 0 : 1);
            int nd = dist[r][c] + w;
            if (nd < dist[nr][nc]) {
                dist[nr][nc] = nd;
                if (w == 0) dq.push_front({nr, nc});
                else dq.push_back({nr, nc});
            }
        }
    }
    cout << dist[n - 1][m - 1] << '\n';
    return 0;
}
```

4.7 Full Python solution

```
from collections import deque
import sys

DIRS = [(1, 0, "D"), (-1, 0, "U"), (0, -1, "L"), (0, 1, "R")]

def main() -> None:
    input = sys.stdin.readline
    n, m = map(int, input().split())
    grid = [input().strip() for _ in range(n)]
    inf = 10**9
    dist = [[inf] * m for _ in range(n)]
    dist[0][0] = 0
    dq = deque([(0, 0)])
    while dq:
        r, c = dq.popleft()
        for dr, dc, ch in DIRS:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < m:
                w = 0 if grid[r][c] == ch else 1
                nd = dist[r][c] + w
                if nd < dist[nr][nc]:
                    dist[nr][nc] = nd
                    if w == 0:
                        dq.appendleft((nr, nc))
                    else:
                        dq.append((nr, nc))
    print(dist[n - 1][m - 1])

if __name__ == "__main__":
    main()
```

5 C. Warp Maze

5.1 Restatement

Walking to a four-neighbor open cell costs 0. Warping to any open cell within Chebyshev distance 2 costs 1. Find the minimum number of warps from S to G .

5.2 Observations

The graph has 0-cost walking edges and 1-cost warp edges. Each cell has at most 29 outgoing transitions: four walks and 25 warp candidates. This is still linear up to a constant.

The state is simply an open cell (r, c) , and $\text{dist}[r][c]$ is the fewest paid warps needed to reach it. Walls never become states. The two action families must remain separate:

Action	Destination	Weight	Deque action after an improvement
walk	one open cardinal neighbor	0	push front
warp	any open cell with $\max(dr , dc) \leq 2$	1	push back

The warp square includes diagonals and also includes the current cell. The latter is harmless: it cannot improve $\text{dist}[r][c]$. It is clearer to iterate all 25 offsets than to introduce a fragile exception. On the sample, free walking explores the start's whole zero-cost component first; one warp then crosses the blocking wall pattern into the component containing G .

5.3 Algorithm

```
find S and G
dist[*][*] = infinity
dist[S] = 0
push S to the front of a deque

while the deque is nonempty:
    (r, c) = pop front
    for each open four-neighbor v of (r, c):
        if dist[r][c] < dist[v]:
            dist[v] = dist[r][c]
            push v to the front
    for every offset dr, dc in [-2, 2]:
        v = (r + dr, c + dc)
        if v is an open in-bounds cell and dist[r][c] + 1 < dist[v]:
```

```

        dist[v] = dist[r][c] + 1
        push v to the back

print -1 if dist[G] is infinity, otherwise dist[G]

```

5.4 Correctness

The modeled graph exactly represents allowed actions and their warp cost. By 0-1 BFS correctness, the computed distance to G is the minimum number of warps.

5.5 Complexity

Each of the $n*m$ cells has at most $4 + 25$ candidate transitions. The constant is fixed, so the running time is $O(n*m)$ and the memory use is $O(n*m)$.

5.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<string> grid(n);
    pair<int,int> start{-1,-1}, goal{-1,-1};
    for (int r = 0; r < n; r++) {
        cin >> grid[r];
        for (int c = 0; c < m; c++) {
            if (grid[r][c] == 'S') start = {r,c};
            if (grid[r][c] == 'G') goal = {r,c};
        }
    }
    const int INF = 1e9;
    vector<vector<int>> dist(n, vector<int>(m, INF));
    deque<pair<int,int>> dq;
    dist[start.first][start.second] = 0;
    dq.push_back(start);
    int wr[4] = {1,-1,0,0}, wc[4] = {0,0,1,-1};
    while (!dq.empty()) {
        auto [r,c] = dq.front(); dq.pop_front();
        int d = dist[r][c];
        for (int i = 0; i < 4; i++) {
            int nr = r + wr[i], nc = c + wc[i];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m ||

```

```

        grid[nr][nc] == '#') {
            continue;
        }
        if (d < dist[nr][nc]) {
            dist[nr][nc] = d;
            dq.push_front({nr,nc});
        }
    }
    for (int dr = -2; dr <= 2; dr++) {
        for (int dc = -2; dc <= 2; dc++) {
            int nr = r + dr, nc = c + dc;
            if (nr < 0 || nr >= n || nc < 0 || nc >= m ||
                grid[nr][nc] == '#') {
                continue;
            }
            if (d + 1 < dist[nr][nc]) {
                dist[nr][nc] = d + 1;
                dq.push_back({nr,nc});
            }
        }
    }
}
int ans = dist[goal.first][goal.second];
cout << (ans == INF ? -1 : ans) << '\n';
return 0;
}

```

5.7 Full Python solution

The Python implementation flattens the grid and surrounds it with a two-cell wall border. Every warp offset then has a valid array index, so the inner loop avoids repeated coordinate bounds checks. A queued distance is stored with its cell; skip a ticket if a later relaxation improved that cell. The deque's 0-1 ordering makes the first non-stale removal of the goal optimal, so it is safe to stop there. An unreachable goal still requires exhausting the search.

```

from collections import deque
import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    n, m = map(int, data[:2])
    rows = data[2:2 + n]
    # Two wall cells around every border make every 5-by-5 warp offset safe.
    stride = m + 4
    border = b'#' * stride
    grid = border * 2 + b''.join(b'##' + row + b'##' for row in rows) + border * 2
    start = goal = -1
    for r, row in enumerate(rows):

```

```

        c = row.find(b'S')
        if c >= 0: start = (r + 2) * stride + c + 2
        c = row.find(b'G')
        if c >= 0: goal = (r + 2) * stride + c + 2
    inf = 10**9
    dist = [inf] * len(grid)
    dist[start] = 0
    dq = deque([(start, 0)])
    walks = (stride, -stride, 1, -1)
    warps = tuple(
        dr * stride + dc
        for dr in range(-2, 3)
        for dc in range(-2, 3)
    )
    while dq:
        cell, distance = dq.popleft()
        if distance != dist[cell]:
            continue
        if cell == goal:
            print(distance)
            return
        for offset in walks:
            other = cell + offset
            if distance < dist[other] and grid[other] != 35:
                dist[other] = distance
                dq.appendleft((other, distance))
        next_distance = distance + 1
        for offset in warps:
            other = cell + offset
            if next_distance < dist[other] and grid[other] != 35:
                dist[other] = next_distance
                dq.append((other, next_distance))
    print(-1)

if __name__ == '__main__':
    main()

```

6 Common wrong answers

- Treating all moves as cost 1.
- Using normal BFS on a graph with zero-cost edges.
- Pushing zero-cost edges to the back.
- Forgetting that warps include diagonal and nearby cells.
- Counting number of moves instead of number of paid actions.

7 Stress testing

For small graphs and grids, compare 0-1 BFS against Dijkstra with a heap. The answers must match. Random tests should include disconnected targets, all-zero paths, and cases where a longer route in steps is cheaper in cost.

8 What to carry forward

0-1 BFS is a specialized shortest-path tool. The hard part is often not the deque; it is modeling the state so that every transition has cost exactly 0 or 1.

9 D. Letter Portals

9.1 Restatement

The grid has ordinary open cells, walls, a start, a goal, and lowercase-letter portal cells. Walking to an open four-neighbor cell costs 1. From a letter cell, jumping to any cell with that same letter costs 0. We need the minimum cost from **S** to **G**, or **-1** when no route exists.

The grid can contain 10^6 cells. It is not enough to use ordinary BFS, because a zero-cost teleport must be processed before a one-cost walk. It is also not enough to scan every matching-letter list every time a portal cell is visited: a grid full of the same letter would become quadratic.

9.2 Model the implicit graph

Use one graph vertex per non-wall grid cell.

```
walk to a four-neighbor open cell    edge cost 1
same-letter teleport                  edge cost 0
```

This is exactly a 0-1 shortest-path problem. Store the positions of each of the 26 letters while reading the grid, then use a deque:

```
cost 0 -> push_front
cost 1 -> push_back
```

9.3 Expanding one portal group once

Suppose the deque first removes a cell labelled **a** at distance **d**. Every other **a** cell is reachable from it for cost 0, so all of them can receive distance **d** immediately. There is no reason to scan the complete **a** list again when another **a** cell is reached later. Mark the letter as expanded after this first scan.

The word *first* matters. In 0-1 BFS, a cell removed from the front has the smallest currently possible distance. Therefore the first removed **a** cell has the minimum distance at which any **a** portal can be reached.

9.4 Walk-through

Consider:

```
S # a # G
. a . a .
. . . . .
```

Walking from S to the left a costs 2:

```
S -> (1,0) -> left a
```

When that a is removed from the deque, expand the a group:

```
left a -- cost 0 --> right a
```

The right a also has distance 2. Two ordinary moves reach G, so the answer is 4. The longer all-walking detour is never preferable.

9.5 Algorithm

```
record positions for each letter while reading the grid
dist[cell] = infinity for every cell
dist[S] = 0
push S into a deque
expanded[letter] = false

while deque is not empty:
    remove cell u from the front

    if u has letter x and expanded[x] is false:
        expanded[x] = true
        for every cell v with letter x:
            relax v using cost 0

    for every open four-neighbor v of u:
        relax v using cost 1

print dist[G], or -1 if it is still infinity
```

To relax an edge of cost w , update only if:

```
dist[u] + w < dist[v]
```

Then push v to the front for $w = 0$, otherwise to the back.

9.6 Correctness proof

We prove that the algorithm prints the minimum possible cost.

Lemma 1. When a portal letter x is expanded, the current distance of the removed x cell is the minimum distance of any route from S to an x cell.

Proof. 0-1 BFS removes cells from the deque in nondecreasing finalized distance order. The expanded cell is the first removed cell labelled x . If a cheaper route to any x cell existed, that cell would have a smaller distance and would be removed first, a contradiction. $\square$

Lemma 2. Expanding each portal letter at most once preserves every shortest-path distance.

Proof. When x is expanded at distance d , every x cell is relaxed to distance at most d by a zero-cost teleport. By Lemma 1, no x cell can later be reached with a value smaller than d . Thus scanning the same zero-cost teleport edges later cannot improve any distance. $\square$

Lemma 3. Apart from skipped redundant portal scans, the algorithm performs the same successful relaxations as 0-1 BFS on the implicit graph.

Proof. Every walking edge is examined whenever its source cell is removed. For a portal edge, its whole same-letter group is examined at the first and minimum-distance source by Lemma 1; Lemma 2 proves later scans are redundant. Each examined edge is relaxed with its stated cost and sent to the correct end of the deque. $\square$

Theorem. The algorithm prints the minimum travel cost from S to G , or -1 exactly when the goal is unreachable.

Proof. By Lemma 3, the algorithm is 0-1 BFS on all useful implicit graph edges. The 0-1 BFS invariant therefore gives the shortest distance from S to every reachable cell, including G . If G remains at infinity, no sequence of valid walking or teleport edges reached it, so it is unreachable. The printed result is consequently correct. $\square$

9.7 Complexity

Each cell has at most four walking edges. Each portal position appears in one letter list, and that list is scanned once. Therefore:

```
Time:    O(n * m)
Memory:  O(n * m)
```

9.8 Implementation notes

- Store letters in 26 lists indexed by `letter - 'a'` or `ord(letter) - ord('a')`.
- Mark a letter expanded before scanning its list.
- A teleport is cost 0, so its improved endpoints go to the deque front.
- A normal grid step costs 1, so it goes to the deque back.
- Do not treat S and G as portal letters; only lowercase characters are portal labels.

9.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<string> grid(n);
    vector<vector<int>> portals(26);
    int start = -1, goal = -1;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];
        for (int c = 0; c < m; ++c) {
            int id = r * m + c;
            char cell = grid[r][c];
            if (cell == 'S') start = id;
            if (cell == 'G') goal = id;
            if ('a' <= cell && cell <= 'z') {
                portals[cell - 'a'].push_back(id);
            }
        }
    }

    const int INF = 1e9;
    vector<int> dist(n * m, INF);
    vector<char> expanded(26, false);
    deque<int> dq;
    dist[start] = 0;
    dq.push_back(start);
    int dr[4] = {1, -1, 0, 0};
    int dc[4] = {0, 0, 1, -1};

    while (!dq.empty()) {
        int id = dq.front();
        dq.pop_front();
        int r = id / m, c = id % m;
        int current = dist[id];

        char cell = grid[r][c];
        if ('a' <= cell && cell <= 'z' && !expanded[cell - 'a']) {
            expanded[cell - 'a'] = true;
            for (int other : portals[cell - 'a']) {
                if (current < dist[other]) {
                    dist[other] = current;
                    dq.push_front(other);
                }
            }
        }
    }
}
```

```

    }
}

for (int k = 0; k < 4; ++k) {
    int nr = r + dr[k], nc = c + dc[k];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m ||
        grid[nr][nc] == '#') {
        continue;
    }
    int other = nr * m + nc;
    if (current + 1 < dist[other]) {
        dist[other] = current + 1;
        dq.push_back(other);
    }
}

cout << (dist[goal] == INF ? -1 : dist[goal]) << '\\\\n';
}

```

9.10 Full Python solution

```

import sys
from collections import deque

def main() -> None:
    data = sys.stdin.buffer.read().split()
    n, m = map(int, data[:2])
    grid = [row.decode() for row in data[2:2 + n]]
    portals = [[] for _ in range(26)]
    start = goal = -1

    for r, row in enumerate(grid):
        for c, cell in enumerate(row):
            cell_id = r * m + c
            if cell == "S":
                start = cell_id
            elif cell == "G":
                goal = cell_id
            elif "a" <= cell <= "z":
                portals[ord(cell) - ord("a")].append(cell_id)

    inf = 10**9
    dist = [inf] * (n * m)
    expanded = [False] * 26
    dist[start] = 0

```

```

dq = deque([start])

while dq:
    cell_id = dq.popleft()
    r, c = divmod(cell_id, m)
    current = dist[cell_id]
    cell = grid[r][c]

    if "a" <= cell <= "z":
        letter = ord(cell) - ord("a")
        if not expanded[letter]:
            expanded[letter] = True
            for other in portals[letter]:
                if current < dist[other]:
                    dist[other] = current
                    dq.appendleft(other)

    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] != "#":
            other = nr * m + nc
            if current + 1 < dist[other]:
                dist[other] = current + 1
                dq.append(other)

print(-1 if dist[goal] == inf else dist[goal])

if __name__ == "__main__":
    main()

```

9.11 Common wrong answers

- Using ordinary BFS, which counts teleports as if they cost one move.
- Putting a zero-cost teleport at the deque back.
- Scanning all cells with letter **a** every time any **a** cell is visited.
- Marking a portal letter used when it is first discovered instead of when it is removed at its minimum distance.
- Allowing moves through walls or treating **S** and **G** as lowercase labels.

9.12 Tests that matter

- no portals, which reduces to an ordinary shortest grid path;
- a portal shortcut that is cheaper than the fewest-step walk;
- repeated cells of one letter, to catch repeated full-list scans;
- an unreachable goal;
- a portal first reached by a costly route but later reachable more cheaply;
- a large open grid, which checks linear memory and time.

10 Stress testing

On a tiny grid, build the implicit graph literally: from every letter cell, add a zero-cost edge to every equal-letter cell. Then run heap Dijkstra and compare its answer with 0-1 BFS. This is deliberately slower but independent of the portal-group optimization. The local generator uses that oracle on small random grids and also includes a 1000 x 1000 open grid with known answer 1998.

11 What to carry forward

0-1 BFS remains linear only when the transition model is also efficient. A large group of equivalent zero-cost moves should usually be activated once at the first finalized distance:

```
implicit zero-cost group + one-time expansion = linear total work
```

First identify the state and the costs, then account for how often every generated transition is scanned.